

Homework 1 - Finding Similar Items

Data Mining

AHMAD AL KHATEEB OMAR ALMASSRI
ahmadak | omaralm@kth.se

November 2025

Overview

This report describes an implementation for finding textually similar documents using *shingling*, *min-hashing*, and *locality-sensitive hashing (LSH)*. The code follows the classic pipeline from Mining of Massive Datasets (MMDS, Ch. 3). The goal is to:

- build sets of hashed k -shingles for each document
- compute exact Jaccard similarity between sets
- construct MinHash signatures
- estimate similarity from signatures, and
- apply LSH banding to propose candidate similar pairs efficiently

1 Task 1

Shingling. Given a document, we construct character shingles of length k . For a string s , the multiset of substrings of length k is

$$\mathcal{S}_k(s) = \{ s[i : i + k] \mid i = 0, \dots, |s| - k \}$$

Each unique shingle is mapped deterministically to an integer (via the Python built-in hashlib, MD5 hash function). A document is represented by the set $A \subset \mathbb{N}$ of its hashed shingles (duplicates removed). The hashed shingle is constructed then by taking the generated MD5-hash value $\bmod 2^{32}$.

2 Task 2

Exact Jaccard on shingle sets. For two documents with shingle sets A and B , the Jaccard similarity is

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

This provides the ground truth for later comparisons with MinHash estimates.

3 Task 3

MinHash signatures. We approximate random permutations with a family of hash functions of the form

$$h_i(x) = (a_i x + b_i) \bmod P, \quad i = 1, \dots, n, \quad (1)$$

where P is a large prime (greater than the shingle-hash universe, i.e., 2^{32}), and (a_i, b_i) are generated once from a fixed seed for reproducibility. The MinHash signature $\sigma(A) \in \mathbb{N}^n$ of a set A is

$$\sigma_i(A) = \min_{x \in A} h_i(x). \quad (2)$$

For two sets A, B , the fraction of matching components

$$\hat{J}(A, B) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\sigma_i(A) = \sigma_i(B)\}$$

is an unbiased estimator of $J(A, B)$, with variance approximately $\text{Var}[\hat{J}] \approx \frac{J(1-J)}{n}$.

4 Task 4

Compare signatures. Given two signatures $\sigma(A)$ and $\sigma(B)$ of equal length n , their similarity estimate is the agreement fraction

$$\text{agree}(\sigma(A), \sigma(B)) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\sigma_i(A) = \sigma_i(B)\}.$$

5 Task 5

Locality-Sensitive Hashing (banding). Let the signature length be $n = b \cdot r$ (split into b bands, r rows per band). Two signatures are considered candidates if they are identical in at least one band (after hashing the band vectors into buckets). Under the standard independence assumptions, if the true similarity is s , the probability of becoming a candidate is

$$\Pr[\text{candidate} \mid s] = 1 - (1 - s^r)^b. \quad (3)$$

A common rule-of-thumb for the S-curve “knee” is

$$t \approx \left(\frac{1}{b}\right)^{1/r}, \quad (4)$$

which we use to choose a reasonable b for a desired threshold t (with $r = n/b$ an integer).

6 Program Execution

Datasets. The dataset used for the execution and result parts are fetched from <https://archive.ics.uci.edu/dataset/113/20+newsgroups>, the UCI Machine Learning Repository (e.g., 20 Newsgroups, Reuters-21578, Enron emails). Download the zip-file and locate it under `./dataset`.

How to run. First, install all the python requirements specified in `requirements.txt` ‘`pip install -r requirements.txt`’. The program can then be run by modifying the file `run_app` (.bat on Windows or .sh on Linux) with the absolute path to the zip-file you downloaded, and run it. This will run with the parameters specified in the batch file. If you want to try other parameters, use the CLI commands below and run the app from the file `main.py`:

```
# Run on all specific parameters
python .\Data_mining_HWs\Homework1\main.py -z <path to the zip file> \
-nod <amount documents to read> \
-k <shingle length> \
-n <signature length> \
-t <threshold> \
-b <amount bands>

# Run on default values
python .\Data_mining_HWs\Homework1\main.py -z <path to the zip file> \
-nod <amount documents to read> \
-k <shingle length> \

# Or just type python .\Data_mining_HWs\Homework1\main.py --help
```

Results

This section presents two aspects of the program, the first one is when it was run for different parameters, and the second one is the scalability.

R1: Parameter Experiment

A total of 27 runs were executed, each one on the same corpus of 1000 documents. The experiments varied the Min-Hash signature length (64, 128, 256), the shingle length ($k = 6, 8, 10$), and the target similarity threshold ($t = 0.6, 0.7, 0.8$). For each configuration, the number of candidate pairs produced by the LSH stage was recorded, together with the corresponding S-curve threshold $(1/b)^{1/r}$ derived from the chosen banding parameters.

Signature length	Shingle length k	Threshold t	Bands b	Avg. candidates	S-curve t (approx.)
256	6-10	0.8	16-32	0	0.65-0.80
256	6-10	0.6-0.7	32	1	0.65
128	6-10	0.8	8	0	0.88
128	6-10	0.6-0.7	16	1	0.70
64	6-10	0.7-0.8	8	0	0.77
64	6-10	0.6	16	1	0.50

Table 1: Summary of observed LSH candidate detections for different parameter settings.

The results show that candidate pairs are retrieved only when the chosen similarity threshold is near or below the analytical S-curve value. For all signature lengths, no candidate appear at $t = 0.8$, while one or more pairs are found once the threshold is lowered to 0.7 or 0.6. This confirms the expected S-curve behavior: LSH begins detecting similar document when the target threshold approaches $(1/b)^{1/r}$. The influence of signature length is minor for this dataset, whereas smaller shingle lengths ($k = 6, 9$) slightly increase the likelihood of overlap.

R2: Scalability (timing vs. corpus size)

In order to obtain insight of how slow the program runs when processing a large amount of documents, figure 1 shows the total runtime measured for corpus sizes ranging from 5 to 1000 documents. The curve increases approximately linearly up to a few hundred documents and then grows more steeply for larger sets. This behavior reflects the rising computational cost

of shingling and Min-Hashing as the number of document increases. ($k=10$, $\text{siglen}=256$, $\text{threshold}=0.8$, $\text{bands}=16$), and repeats 1 time.

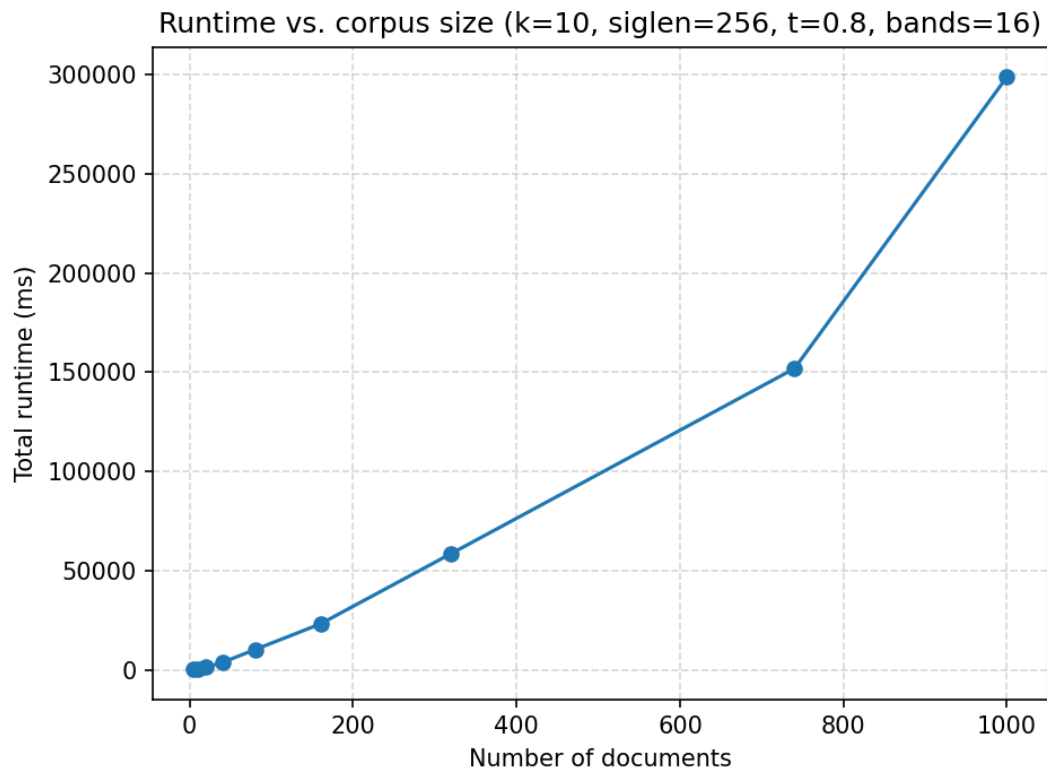


Figure 1: Runtime vs. corpus size for ($k = 10$, $\text{siglen} = 256$, $\text{threshold} = 0.8$, $\text{bands} = 16$).